

Project Post-Mortem: EC2 CI/CD Pipeline

CodeCommit → CodeBuild → CodeArtifact → Manual Approval → CodeDeploy (EC2)

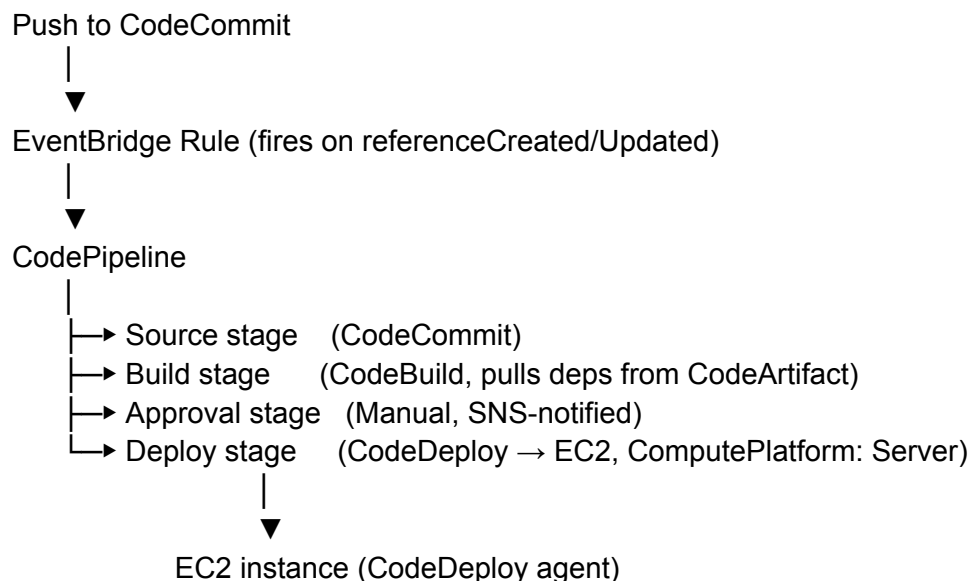
Prepared as a debugging journey report — written the way a senior engineer would document an incident, for the purpose of building exam-ready intuition for AWS Certified Developer – Associate (DVA-C02).

Why this report exists

Passing DVA-C02 at a 900+ level isn't about memorizing service names — it's about recognizing *failure signatures*. The exam loves to hand you an error message or a symptom and ask "what's the root cause and what's the fix?" That is **exactly** what this project put you through, for real, six times in a row. This report reconstructs the whole incident chain in order, the way you'd write it up for a team retro, so the pattern-matching sticks.

Every section follows the same structure a senior engineer uses when triaging: **Symptom** → **Where it actually failed** → **Root cause** → **Fix** → **Why this matters for the exam**.

The system, in one diagram



appspec.yml lifecycle hooks:
ApplicationStop → DownloadBundle → BeforeInstall
→ Install → AfterInstall → ApplicationStart → ValidateService
↓
▼
systemd service (myapp.service) → Node/Express app

Six failures happened across **four different layers** of this stack: CloudFormation resource modeling, IAM permissions, OS-level process configuration, and Linux security primitives. That spread is not an accident — it's representative of what actually breaks in AWS deployments, and it's why DVA-C02 tests all four layers separately.

Incident 1 — CodeArtifact domain name exceeded 50 characters

Symptom (CloudFormation console):

```
CREATE_FAILED — ArtifactRepository
Model validation failed (/DomainName: 2 schema violations found)
#/DomainName: expected maxLength: 50, actual: 68
#/DomainName: failed validation constraint for keyword [pattern]
```

Where it actually failed: Resource creation, very early — before any compute or networking existed. This is a *template validation* failure, not a runtime failure.

Root cause: The `DomainName` property on `AWS::CodeArtifact::Repository` was built from `${ProjectName}-domain`, and `ProjectName` had no length constraint. A long parameter value silently produced an invalid domain name string.

The fix: Added explicit `MinLength`, `MaxLength`, and `AllowedPattern` constraints directly on the CloudFormation `Parameter` block:

```
ProjectName:
  Type: String
  MinLength: 3
  MaxLength: 20
  AllowedPattern: '^[a-z][a-z0-9-]{1,18}[a-z0-9]$'
```

Senior takeaway: Whenever a resource property is *derived* from a parameter via `!Sub`, the parameter needs its own constraints — CloudFormation won't infer them from the downstream resource's schema. Fail fast at the parameter level, not deep in resource creation.

Exam relevance (Domain 1 – Deployment): DVA-C02 tests CloudFormation parameter validation (`AllowedPattern`, `AllowedValues`, `MinLength/MaxLength`, `ConstraintDescription`) as a first-class topic. Expect a question describing a stack failing on a naming/length rule and asking which parameter property prevents it.

Incident 2 — Identical error persisted after the "fix" (the real trap)

Symptom: Same exact error, same "actual: 68" character count, on a second deploy attempt — *even after* the `ProjectName` length constraint was added.

Where it actually failed: Same resource, same property — but the constraint fix didn't touch the actual bug.

Root cause — the important one: `!Ref` on `AWS::CodeArtifact::Domain` returns the **ARN**, not the domain name. This is *inconsistent* with most CloudFormation resources (S3 buckets, CodeBuild projects, CodeDeploy applications all return their *name* on `!Ref`). The line:

```
DomainName: !Ref ArtifactDomain
```

was silently resolving to

`arn:aws:codeartifact:us-east-1:<account>:domain/my-ec2-app-domain` — an ARN, which is both too long and fails the domain-name pattern (contains `:` and `/`).

The fix: Use `!GetAtt ArtifactDomain.Name` instead of `!Ref` — and the same correction was needed in three separate places that all referenced the domain/repository (the `ArtifactRepository` resource, the `CODEARTIFACT_DOMAIN/CODEARTIFACT_REPO` CodeBuild environment variables, and a stack Output).

Senior takeaway: Never assume `!Ref` behavior is uniform across resource types. Always check the CloudFormation resource reference page's "Ref" return-value section for anything you haven't used before — especially in service families like CodeArtifact, CodePipeline actions, and IAM, where `Ref` frequently returns an ARN instead of a friendly name.

Exam relevance (Domain 1 – Deployment): This is a classic DVA-C02 trap question format: "What does `!Ref` return for resource type X?" Memorize the exceptions, don't assume

consistency. Also tests `!GetAtt` as the correct escape hatch when `!Ref` gives you the wrong shape of data.

Incident 3 — `git clone` "already exists" loop (tooling, not AWS)

Symptom: Repeated `fatal: destination path already exists and is not an empty directory`, followed by `cp` commands failing with `No such file or directory`.

Where it actually failed: Local shell navigation — no AWS resource was involved at all.

Root cause: Two nested folders shared the same name (`my-ec2-app-repo/my-ec2-app-repo/`), and repeated `cd ..` commands overshot the correct directory level, so `cp` was being run from a parent directory that didn't contain the source files.

The fix: Explicit `cd` to the *full absolute path*, `ls` as a sanity check before every `cp`, and a recommendation to rename one of the duplicate-named folders to avoid the ambiguity going forward.

Senior takeaway: Not every "AWS project" failure is an AWS problem. A disciplined debugging habit — `pwd` and `ls` before any destructive or copy operation — would have caught this in 10 seconds instead of three failed attempts. This is the human-tooling equivalent of "read the error message fully before acting."

Exam relevance: DVA-C02 doesn't test shell navigation directly, but it *does* test CodeCommit/Git workflows (credential helpers, `git-remote-codecommit`, HTTPS vs SSH, IAM policy requirements for CodeCommit access). Worth knowing cold: `git clone codecommit::<region>://<repo>` requires `git-remote-codecommit` installed and IAM credentials configured, not an SSH key.

Incident 4 — Approval CLI call failed on parameter parsing

Symptom:

ParamValidation: Invalid type for parameter result.summary, value: ['Approved', 'deploying to EC2'], type: <class 'list'>, valid types: <class 'str'>

Root cause: The AWS CLI's *shorthand syntax* (`--result summary="...", status=Approved`) uses commas as field separators. A comma inside the summary text itself got parsed as a delimiter, turning a string into an accidental list.

The fix: Either remove the comma, or switch to explicit JSON input format for the `--result` flag:

```
--result '{"summary": "Approved, deploying to EC2", "status": "Approved"}'
```

Senior takeaway: AWS CLI shorthand syntax is convenient but fragile for any free-text field. Once a value might contain punctuation, switch to `--cli-input-json` or inline JSON rather than fighting shorthand escaping.

Exam relevance (Domain 1 – Deployment): DVA-C02 tests the CodePipeline `put-approval-result` API and the manual approval action's required fields (`summary`, `status: Approved|Rejected`, and the approval `token`, which is single-use and expires). Know that approval tokens are tied to a specific `actionExecutionId` and can't be reused across executions.

Incident 5 — CodeDeploy `DownloadBundle` failed: missing `s3:GetObjectVersion`

Symptom:

HEALTH_CONSTRAINTS — too many individual instances failed deployment...

This top-level error is deliberately generic. The *real* error was buried one level deeper, in the CodeDeploy agent log:

```
User: .../my-ec2-app-ec2-instance-role/... is not authorized to perform:
s3:GetObjectVersion on resource: "...BuildOutput/8QFCBqx" because no
identity-based policy allows the s3:GetObjectVersion action
```

Where it actually failed: `DownloadBundle` — the very first lifecycle event, before any of your `appspec.yml` scripts even ran. This is why `BeforeInstall` through `ValidateService` all showed `Skipped`: CodeDeploy never got far enough to attempt them.

Root cause: The pipeline's `ArtifactBucket` had **S3 versioning enabled**. When an S3 bucket is versioned, CodePipeline stores and references build artifacts by a specific object **version ID**. Retrieving a specific version requires the IAM action `s3:GetObjectVersion` — which is *distinct* from plain `s3:GetObject`. The EC2 instance role only granted the latter.

The fix:

- Effect: Allow
- Action:
 - `s3:GetObject`
 - `s3:GetObjectVersion`
- Resource:
 - `!Sub '${ArtifactBucket.Arn}/*'`

Senior takeaway: `HEALTH_CONSTRAINTS` is a *symptom*, not a diagnosis — CodePipeline surfaces it whenever any instance in a deployment group fails, for almost any reason. The actual root cause always lives one layer deeper: the CodeDeploy deployment's `errorInformation`, the per-instance `lifecycleEvents`, or (deepest of all) the agent log on the box itself (`/var/log/aws/codedeploy-agent/codedeploy-agent.log`). Senior debugging discipline is: **never stop at the first error message — walk down the stack until you find the one with a specific resource ARN and action name in it.**

Exam relevance (Domain 2 – Security, and Domain 4 – Troubleshooting): This is one of the highest-yield exam patterns. DVA-C02 loves scenarios where a versioned S3 bucket is involved and a role only has `s3:GetObject`. Memorize: **versioned bucket + specific version retrieval = `s3:GetObjectVersion` required**, separate from `s3:GetObject`. Also expect CodeDeploy lifecycle event ordering to be tested directly: `ApplicationStop` → `DownloadBundle` → `BeforeInstall` → `Install` → `AfterInstall` → `ApplicationStart` → `ValidateService`.

Incident 6a — App running, but nothing on port 80 (**EACCES**)

Symptom: CodeDeploy succeeded, `systemctl status myapp` showed `active (running)`, but `curl http://localhost/health` returned `Connection refused`.

Root cause: `journalctl -u myapp` showed the real error immediately:

Error: listen EACCES: permission denied 0.0.0.0:80

Linux reserves ports below 1024 for root by default. The systemd unit ran the process as `User=ec2-user`, a non-privileged user, so Node's `app.listen(80)` was denied at the OS level — this has nothing to do with AWS at all.

Senior takeaway: "The service shows as running" and "the service is actually working" are different claims. `systemctl status` only tells you the process didn't immediately crash — it says nothing about whether the application is functioning. Always verify the actual behavior (`curl`, a health check, a real request) rather than trusting process status alone.

Exam relevance: Not directly DVA-C02 (this is closer to SysOps/OS fundamentals), but it reinforces a recurring theme across all AWS deploy/monitor questions: **"deployed successfully" and "healthy" are not the same signal**, which is exactly why CodeDeploy has a distinct `ValidateService` lifecycle hook — its entire purpose is to catch exactly this gap.

Incident 6b — `setcap` failed with "Invalid file 'setcap' for capability operation"

Root cause: The chosen fix for Incident 6a was to grant the Node binary the `cap_net_bind_service` capability instead of running the whole process as root (least-privilege principle: grant the one specific permission needed, not blanket root access). But `setcap` cannot operate on symlinks — and on Amazon Linux 2023, `/usr/bin/node` is a symlink managed by the `alternatives` system (visible in the install log: *"registered node-18 in the alternatives"*).

The fix: Resolve the real underlying binary path first, then apply the capability there:

```
sudo setcap 'cap_net_bind_service=+ep' "$(readlink -f /usr/bin/node)"
```

Senior takeaway: Two compounding lessons in one incident:

1. **Prefer capabilities over `sudo/root`** when a process needs one specific elevated permission — this is the same least-privilege principle IAM teaches, just at the OS layer.
2. **Symlinks break tools that operate on file metadata/inodes** (`setcap`, `chmod` in some contexts, extended attributes generally). When a system-level tool gives a cryptic error about "invalid file," check whether you're pointing it at a symlink instead of the real file.

Exam relevance: Outside DVA-C02's direct scope, but the underlying principle — grant the minimum specific permission rather than broad elevated access — is the same philosophy

tested constantly in IAM policy questions (least privilege, scoped resource ARNs instead of *, condition keys instead of blanket allows).

The meta-pattern across all six incidents

#	Layer	First error shown	Real root cause	Gap between the two
1	CloudFormation schema	<code>maxLength: 50, actual: 68</code>	Unconstrained parameter	Shallow — visible immediately
2	CloudFormation <code>Ref</code> semantics	<code>same error, unchanged</code>	<code>!Ref</code> returns ARN not name	Deep — required knowing resource-specific <code>Ref</code> behavior
3	Local tooling	<code>already exists</code>	Wrong working directory	Shallow — <code>pwd/ls</code> would reveal it
4	CLI parsing	<code>ParamValidation</code> type error	Comma broke shorthand syntax	Shallow — but easy to misdiagnose as an AWS-side bug
5	IAM	<code>HEALTH_CONSTRAINTS</code> (generic)	Missing <code>s3:GetObjectVersion</code>	Very deep — required agent logs, not the pipeline console
6	OS/Linux	<code>Connection refused</code>	<code>EACCES</code> on port 80, then symlink breaking <code>setcap</code>	Deep — required <code>journalctl</code> , then domain knowledge of <code>alternatives</code>

The single biggest skill this project exercised, repeatedly, is refusing to accept the first error message as the root cause. Every genuinely hard incident here (2, 5, 6) had a generic or misleading top-level error that only resolved once you went one or two layers deeper — CloudFormation docs, agent logs, `journalctl`. That instinct — **the error message you're shown is a symptom; go find the log that names a specific resource and a specific denied action** — is the core skill DVA-C02's troubleshooting domain is built to test, and it's the skill that separates a 700 scorer from a 900 scorer on scenario questions.

Command reference — what to reach for, in order, next time something fails

1. CloudFormation stack-level failure

```
aws cloudformation describe-stack-events --stack-name <name> \
  --query "StackEvents[?ResourceStatus=='CREATE_FAILED']"
```

2. CodePipeline stage/action status

```
aws codepipeline get-pipeline-state --name <pipeline-name>
```

3. CodeDeploy deployment-level error (still often generic)

```
aws deploy get-deployment --deployment-id <id> \
  --query "deploymentInfo.{status:status, errorInfo:errorInformation}"
```

4. CodeDeploy per-instance lifecycle events (this is where the real error usually is)

```
aws deploy list-deployment-instances --deployment-id <id>
aws deploy get-deployment-instance --deployment-id <id> --instance-id <instance-id>
```

5. On the instance itself — the CodeDeploy agent's own log

```
sudo tail -100 /var/log/aws/codedeploy-agent/codedeploy-agent.log
```

6. On the instance — the actual application process log

```
sudo journalctl -u myapp -n 50 --no-pager
```

7. Basic reachability check — is anything even listening?

```
curl -v http://localhost/health
sudo ss -tlnp | grep :80
```

Notice the direction: **pipeline** → **deployment** → **instance** → **agent** → **process**. Every incident in this project followed exactly this path from a vague top-level status down to a specific, fixable line.

What to drill for the exam specifically

- **Ref vs GetAtt return values** per resource type — don't assume; know the exceptions (CodeArtifact, IAM roles' **Ref** returns name but **GetAtt Arn** needed for policies, etc.)
- **CodeDeploy lifecycle hook order** for EC2/on-premises (**Server** compute platform) — cold recall, including which hooks run as which user and which can/can't be skipped
- **s3:GetObject vs s3:GetObjectVersion** — triggered specifically by bucket versioning being enabled

- **CodePipeline artifact store requirements** — S3 bucket is mandatory, and IAM roles for every stage actor (Pipeline, Build, Deploy) need distinct, scoped permissions
- **Manual approval actions** — `put-approval-result`, token single-use semantics, SNS integration
- **EventBridge vs polling triggers** for CodeCommit → CodePipeline
- **CloudFormation parameter constraints** (`AllowedPattern`, `MinLength/MaxLength`, `ConstraintDescription`) as a first line of defense against invalid resource properties

If you can look at a DVA-C02 scenario question and immediately ask "what's the *generic* error being shown, and where would the *specific* error actually live" — you've internalized the exact instinct this project just gave you six times over.